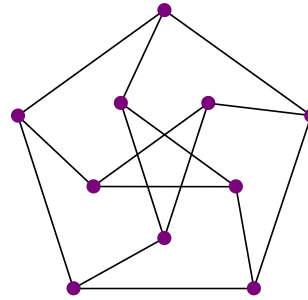




Universidad de Sevilla
SQLito

Hecho por Fernando Giráldez

27 de septiembre de 2026

Índice

1. Conceptos básicos de Python	3
1.1. Ejecutar un programa	3
1.2. Variables y tipos	3
1.3. Operadores	3
1.4. Condicionales	3
1.5. Bucles	4
1.6. Funciones	4
1.7. Errores comunes	4
1.8. Complejidad según n	5
2. Entrada y salida	6
2.1. Leer números	6
2.2. Varios casos de prueba	6
2.3. Leer hasta el final del fichero	6
2.4. Leer todo de golpe	6
2.5. Leer una cuadrícula de caracteres	6
2.6. Leer una matriz de números	6
2.7. Imprimir	6
3. Estructuras de datos	8
3.1. Lista	8
3.2. Lista 2D (matriz)	8
3.3. Diccionario	8
3.4. Conjunto (set)	8
3.5. Counter (contador)	9
3.6. Pila	9
3.7. Cola	9
3.8. Montículo (cola de prioridad)	9
4. Técnicas	10
4.1. Funciones integradas útiles	10
4.2. Ordenar	10
4.3. Sumas prefijas	10
4.4. Búsqueda binaria	10
4.5. Fuerza bruta	10
5. Matemáticas	12
5.1. MCD y MCM	12
5.2. ¿Es primo?	12
5.3. Divisores	12
5.4. Criba de Eratóstenes	12
5.5. Dígitos y binario	12
6. Cadenas de texto	14
6.1. Lo básico de los textos	14

6.2. Caracteres y códigos	14
6.3. Palíndromo	14
6.4. Anagrama	14
7. Grafos	15
7.1. Leer un grafo	15
7.2. Leer un grafo con pesos	15
7.3. BFS (camino más corto)	15
7.4. DFS (alcanzabilidad)	15
7.5. Componentes conexas	15
7.6. BFS en una cuadrícula	16

1. Conceptos básicos de Python

1.1. Ejecutar un programa

Guarda el código como `sol.py` y pruébalo con `python3 sol.py < in.txt`. Envíalo con **PyPy 3** si el juez lo ofrece (mucho más rápido); si no, con **Python 3**.

- Los bloques se definen con la sangría: toda línea que acaba en `:` va seguida de líneas con 4 espacios más. Nunca mezcles tabuladores y espacios.
- Lee exactamente el formato de entrada e imprime exactamente la salida pedida. Nunca imprimas mensajes como "Introduce n:".
- Prueba con el ejemplo del enunciado antes de enviar, y también con casos pequeños y extremos ($n = 1$, n máximo).

1.2. Variables y tipos

Una variable es un nombre para un valor; no hace falta declararla. Tipos principales: `int` (entero), `float` (decimal), `str` (texto), `bool` (`True/False`) y `None` (nada). Se convierten con `int()`, `float()`, `str()`.

```
variables.py
age = 20
price = 9.99
name = "Ana"
is_ok = True
nothing = None
print(type(age), type(price), type(name), type(is_ok))

x = int("42")
y = float("3.5")
s = str(123)
b = bool(0)

a, c = 1, 2
a, c = c, a
print(a, c)
```

1.3. Operadores

`/` siempre devuelve un `float`; usa `//` para la división entera (redondea hacia abajo, también con negativos). `%` es el resto. Los enteros nunca se desbordan. `pow(a, b, m)` calcula $a^b \bmod m$ rápido (típico $m = 10^9 + 7$). `round` redondea los `.5` al par: `round(2.5) == 2`. La lógica usa `and` (y), `or` (o), `not` (no).

```
operators.py
a, b = 17, 5
print(a + b, a - b, a * b)
print(a / b)
print(a // b, a % b)
print(-17 // 5, -17 % 5)
print(a ** 2, 2 ** 100)
print(pow(a, b, 1000))
print(a == b, a != b, a < b, a >= b)
print(a > 0 and b > 0, a > 9 or b > 9, not a)
print(abs(-3), round(2.5), round(3.14159, 2))

x = 10
x += 5
x //= 2
print(x)
```

1.4. Condicionales

`if` (si), `elif` (si no, si), `else` (si no). Se compara con `==` (no con `=`, que asigna). Se pueden encadenar comparaciones como `1 <= n <= 10`.

```
conditionals.py
n = int(input())

if n > 0:
    print("positive")
elif n < 0:
    print("negative")
else:
    print("zero")

if 1 <= n <= 10 and n % 2 == 0:
    print("small even")

parity = "even" if n % 2 == 0 else "odd"
print(parity)
```

1.5. Bucles

for repite para cada valor; while repite mientras se cumpla la condición. range(a, b, paso) va desde a hasta b - 1: b no se incluye. range(n) es 0, ..., $n - 1$. break sale del bucle y continue salta a la siguiente vuelta.

```
loops.py

for i in range(5):
    print(i, end=" ")
print()

for i in range(2, 10, 3):
    print(i, end=" ")
print()

for i in range(5, 0, -1):
    print(i, end=" ")
print()

total = 0
for x in [3, 1, 4]:
    total += x

n = 3700
digits = 0
while n > 0:
    n //= 10
    digits += 1

for i in range(10):
    if i == 3:
        continue
    if i == 6:
        break
    print(i, end=" ")
print()
```

1.6. Funciones

Defínelas con def antes de llamarlas. return puede devolver varios valores a la vez (una tupla). La recursión está limitada a unos 1000 niveles: mejor usa bucles, o añade sys.setrecursionlimit(10**6) después de import sys.

```
functions.py

def add(a, b):
    return a + b

def greet(name="world"):
    return f"Hello, {name}!"

def min_max(v):
    return min(v), max(v)

def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

lo, hi = min_max([4, 2, 9])
print(add(2, 3), greet(), greet("Ana"), lo, hi)
```

1.7. Errores comunes

Error	Causa habitual
IndentationError	Espacios mal puestos, falta sangría tras :
NameError	Nombre mal escrito, o usado antes de asignarlo
TypeError	"a" + 1: usa str(1) o un f-string
ValueError	int("3 4"): falta el .split()
IndexError	v[len(v)]: los índices válidos son 0.. $n - 1$
KeyError	Clave que no está en el dict: usa d.get(k, 0)
ZeroDivisionError	Dividir o hacer % entre cero
RecursionError	Recursión demasiado profunda: usa un bucle

E0Error	Más <code>input()</code> que líneas tiene la entrada
Time limit	Lectura lenta o demasiadas operaciones
Wrong answer	Formato de salida, espacios, casos límite

1.8. Complejidad según n

Python hace unas 10^7 operaciones simples por segundo (PyPy unas 10 veces más):

n máximo	Complejidad	Idea típica
$n \leq 10$	$O(n!)$	Todas las permutaciones
$n \leq 20$	$O(2^n)$	Todos los subconjuntos
$n \leq 200$	$O(n^3)$	Tres bucles anidados
$n \leq 3 \cdot 10^3$	$O(n^2)$	Dos bucles anidados
$n \leq 2 \cdot 10^5$	$O(n \log n)$	Ordenar, búsqueda binaria
$n \leq 10^6$	$O(n)$	Un bucle, sumas prefijas
$n \leq 10^{12}$	$O(\sqrt{n})$	Divisores, primalidad
$n > 10^{12}$	$O(\log n)$ o $O(1)$	Fórmula matemática

2. Entrada y salida

2.1. Leer números

`input()` lee una línea entera como texto. `.split()` la corta por los espacios y `map(int, ...)` convierte cada trozo en entero. Cada `input()` consume exactamente una línea.

read-numbers.py

```
n = int(input())
a, b = map(int, input().split())
v = list(map(int, input().split()))
x = float(input())
word = input().strip()
```

2.2. Varios casos de prueba

La primera línea es el número de casos t . Lee todas las líneas de cada caso, aunque ya sepas la respuesta.

read-test-cases.py

```
t = int(input())
for _ in range(t):
    n = int(input())
    v = list(map(int, input().split()))
    print(sum(v))
```

2.3. Leer hasta el final del fichero

Cuando no te dicen cuántas líneas hay, recorre `sys.stdin`.

read-until-eof.py

```
import sys

for line in sys.stdin:
    a, b = map(int, line.split())
    print(a + b)
```

2.4. Leer todo de golpe

La forma más rápida, útil cuando la entrada es muy grande ($> 10^5$ líneas). `data` es la lista de todas las palabras de la entrada, sin importar los saltos de línea.

read-all.py

```
import sys

data = sys.stdin.read().split()
n = int(data[0])
v = list(map(int, data[1:n + 1]))
```

2.5. Leer una cuadrícula de caracteres

`grid[r][c]` es la fila r , columna c , empezando ambas en 0. Los textos no se pueden modificar: usa `[list(input().strip()) for _ in range(n)]` si necesitas cambiar celdas.

read-grid.py

```
n, m = map(int, input().split())
grid = [input().strip() for _ in range(n)]
print(grid[0][1])
```

2.6. Leer una matriz de números

read-matrix.py

```
n, m = map(int, input().split())
a = [list(map(int, input().split())) for _ in range(n)]
print(a[n - 1][m - 1])
```

2.7. Imprimir

`print` separa sus argumentos con un espacio y termina con un salto de línea; se cambia con `sep=` y `end=`. `f"{x:.2f}"` imprime 2 decimales. Para muchas líneas, guárdalas en una lista e imprímelas de una vez con `"\n".join`.

output.py

```
v = [1, 2, 3]
print(*v)
print(" ".join(map(str, v)))
print(*v, sep="\n")
print("a", "b", sep="")
print("no newline", end="")
print()

x = 3.14159
print(f"{x:.2f}")
```

```
print(f"Case #{1}: {x:.3f}")  
print("YES" if len(v) > 2 else "NO")  
  
out = [str(i * i) for i in range(4)]  
print("\n".join(out))
```

3. Estructuras de datos

3.1. Lista

Secuencia ordenada, con índices de 0 a $\text{len}(v) - 1$; $v[-1]$ es el último. $v[a:b]$ es un trozo de a a $b - 1$. `append` y `pop()` son rápidos; `insert(0, x)`, `pop(0)`, `remove`, `in` e `index` recorren toda la lista.

list.py

```
v = [5, 3, 8]
v.append(1)
v.insert(0, 9)
last = v.pop()
first = v.pop(0)
v.remove(3)
print(len(v), v[0], v[-1])
print(v[1:], v[::-1])
print(8 in v, v.index(8), v.count(5))

zeros = [0] * 5
squares = [i * i for i in range(5)]
evens = [x for x in range(10) if x % 2 == 0]

for i, x in enumerate(v):
    print(i, x)
```

3.2. Lista 2D (matriz)

Nunca la crees con $[[0] * m] * n$: todas las filas son la misma lista, y al cambiar una cambian todas.

list-2d.py

```
n, m = 3, 4
good = [[0] * m for _ in range(n)]
bad = [[0] * m] * n
good[0][0] = 1
bad[0][0] = 1
print(good[1][0], bad[1][0])

rows, cols = len(good), len(good[0])
for r in range(rows):
```

```
for c in range(cols):
    good[r][c] = r * cols + c
```

3.3. Diccionario

Asocia claves con valores y las busca rápido. Leer una clave que no existe con `d[k]` da `KeyError`; usa `d.get(k, por_defecto)`. Las claves deben ser inmutables (números, textos, tuplas).

dict.py

```
age = {"ana": 20, "luis": 25}
age["eva"] = 30
age["ana"] += 1
print(age["ana"], len(age))
print(age.get("bob", 0), "luis" in age)
del age["luis"]
for name, a in age.items():
    print(name, a)

count = {}
for w in "a b a c a".split():
    count[w] = count.get(w, 0) + 1
```

3.4. Conjunto (set)

Colección sin orden y sin repetidos; `x in s` es rápido (en listas no). `{}` es un diccionario vacío: un conjunto vacío es `set()`.

set.py

```
s = {3, 1, 4}
s.add(1)
s.add(5)
s.discard(3)
print(len(s), 4 in s, 3 in s)

a, b = {1, 2, 3}, {2, 3, 4}
print(a | b, a & b, a - b)

unique = len(set([1, 1, 2, 3, 3]))
empty = set()
```

3.5. Counter (contador)

Un diccionario que cuenta cuántas veces aparece cada elemento. Las claves que no están cuentan 0.

```
counter.py
from collections import Counter

c = Counter("banana")
print(c["a"], c["z"])
print(c.most_common(2))

v = Counter([3, 1, 3, 2, 3])
print(max(v, key=v.get))
```

3.6. Pila

El último en entrar es el primero en salir. Es una lista normal: `append` apila, `pop()` quita la cima, `stack[-1]` la mira y `not stack` dice si está vacía. Ejemplo: paréntesis balanceados.

```
stack.py
def balanced(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif not stack or stack[-1] != pairs[ch]:
            return False
        else:
            stack.pop()
    return not stack
```

3.7. Cola

El primero en entrar es el primero en salir. Usa deque: `popleft` es rápido, mientras que `list.pop(0)` es lento.

```
queue.py
from collections import deque

q = deque()
```

```
q.append(1)
q.append(2)
front = q[0]
x = q.popleft()
q.appendleft(0)
y = q.pop()
print(x, y, len(q))
```

3.8. Montículo (cola de prioridad)

Da el menor elemento rápido: `h[0]` lo mira y `heappop` lo saca. Para el mayor, mete los valores en negativo. Las tuplas se comparan por el primer elemento y, si empatan, por el segundo.

```
heap.py
import heapq

h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
smallest = h[0]
x = heapq.heappop(h)

v = [4, 7, 2]
heapq.heapify(v)

big = []
for y in [5, 9, 2]:
    heapq.heappush(big, -y)
largest = -heapq.heappop(big)
```

4. Técnicas

4.1. Funciones integradas útiles

sum, min, max y len recorren la lista una vez. enumerate(v) da pares (índice, valor) y zip(a, b) recorre dos listas a la vez.

built-ins.py

```
v = [4, 1, 7, 3]
print(sum(v), min(v), max(v), len(v))
print(sum(v) / len(v))
print(any(x > 5 for x in v), all(x > 1 for x in v))
print(list(range(3)), list(reversed(v)))

names = ["ana", "bo"]
for name, x in zip(names, v):
    print(name, x)
```

4.2. Ordenar

v.sort() cambia v; sorted(v) devuelve una lista nueva. $O(n \log n)$. Las tuplas se ordenan por el primer elemento y, si empatan, por el segundo. key dice qué comparar; pon un número en negativo para ordenarlo de mayor a menor.

sort.py

```
v = [5, 2, 9, 1]
v.sort()
w = sorted(v, reverse=True)

words = ["pear", "fig", "banana"]
by_len = sorted(words, key=len)

pairs = [(2, "b"), (1, "z"), (2, "a")]
pairs.sort()

people = [("ana", 30), ("bo", 25), ("cy", 30)]
people.sort(key=lambda p: (-p[1], p[0]))
```

4.3. Sumas prefijas

p[i] es la suma de los i primeros elementos. Tras preparar en $O(n)$, range_sum(p, l, r) da la suma de v[l..r] (ambos incluidos, desde 0) en $O(1)$.

prefix-sums.py

```
def prefix_sums(v):
    p = [0] * (len(v) + 1)
    for i, x in enumerate(v):
        p[i + 1] = p[i] + x
    return p

def range_sum(p, l, r):
    return p[r + 1] - p[l]
```

4.4. Búsqueda binaria

v debe estar ordenada. bisect_left(v, x) es la primera posición con $v[i] \geq x$ y bisect_right(v, x) la primera con $v[i] > x$. $O(\log n)$. count_between cuenta los valores en [lo, hi] y necesita $lo \leq hi$.

binary-search.py

```
from bisect import bisect_left, bisect_right

def contains(v, x):
    i = bisect_left(v, x)
    return i < len(v) and v[i] == x

def count_between(v, lo, hi):
    return bisect_right(v, hi) - bisect_left(v, lo)
```

4.5. Fuerza bruta

Si n es pequeño, prueba todas las opciones. permutations da los $n!$ órdenes, combinations(v, k) las formas de elegir k elementos y product(..., repeat=n) las 2^n elecciones de coger o no cada uno.

brute-force.py

```
from itertools import combinations, permutations, product

v = [1, 2, 3]
print(list(permutations(v)))
```

```
print(list(combinations(v, 2)))
print(list(product([0, 1], repeat=2)))

def subsets_with_sum(v, target):
    count = 0
    for take in product([False, True], repeat=len(v)):
        s = sum(x for x, t in zip(v, take) if t)
        count += s == target
    return count
```

5. Matemáticas

5.1. MCD y MCM

Usa `math.gcd` (máximo común divisor) y `math.lcm` (mínimo común múltiplo, Python 3.9+). La versión a mano es el algoritmo de Euclides.

```
gcd-lcm.py
import math

print(math.gcd(12, 18), math.lcm(4, 6))

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a // gcd(a, b) * b
```

5.2. ¿Es primo?

Solo prueba divisores hasta $\sqrt{n}$: $O(\sqrt{n})$. 0 y 1 no son primos.

```
is-prime.py
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

5.3. Divisores

Todos los divisores de $n \geq 1$ en orden creciente, en $O(\sqrt{n})$.

```
divisors.py
def divisors(n):
    small, big = [], []
    i = 1
    while i * i <= n:
        if n % i == 0:
            small.append(i)
            if i != n // i:
                big.append(n // i)
        i += 1
    return small + big[::-1]
```

5.4. Criba de Eratóstenes

`prime[i]` dice si i es primo para todo $0 \leq i \leq n$, con $n \geq 1$. Úsala cuando necesites muchos primos.

```
sieve.py
import math

def sieve(n):
    prime = [False, False] + [True] * (n - 1)
    for i in range(2, math.isqrt(n) + 1):
        if prime[i]:
            for j in range(i * i, n + 1, i):
                prime[j] = False
    return prime
```

5.5. Dígitos y binario

Convertir el número en texto es la forma más fácil de trabajar con sus dígitos. Para $n \geq 0$.

```
digits.py
def digit_sum(n):
    return sum(int(d) for d in str(n))

def reverse_number(n):
    return int(str(n)[::-1])
```

```
def to_binary(n):  
    return bin(n)[2:]
```

```
def from_binary(s):  
    return int(s, 2)
```

6. Cadenas de texto

6.1. Lo básico de los textos

Los textos (str) no se pueden modificar: los métodos devuelven un texto nuevo. Para cambiar caracteres, conviértelo en lista y vuelve a unirlo con "".join. find devuelve -1 si no lo encuentra. No construyas un texto largo con += en un bucle: añade a una lista y únala.

```
string.py

s = "Hello World"
print(len(s), s[0], s[-1], s[0:5])
print(s.lower(), s.upper())
print(s.split())
print(s.replace("l", "L"))
print(s.find("World"), s.find("xyz"), "lo" in s)
print(s.count("o"), s[::-1])
print("-".join(["a", "b", "c"]))
print("123".isdigit(), "abc".isalpha(), "A".isupper())

chars = list(s)
chars[0] = "J"
t = "".join(chars)
```

6.2. Caracteres y códigos

ord da el número de un carácter y chr hace lo contrario. ord(c) - ord("a") es la posición de una letra minúscula (a = 0). shift es un cifrado César sobre minúsculas.

```
ord-chr.py

print(ord("a"), ord("A"), ord("0"))
print(chr(97), chr(ord("a") + 2))
pos = ord("d") - ord("a")

def shift(ch, k):
    return chr((ord(ch) - ord("a") + k) % 26 + ord("a"))
```

6.3. Palíndromo

s[::-1] es s al revés. Distingue mayúsculas: usa antes s.lower() si hace falta.

```
palindrome.py

def is_palindrome(s):
    return s == s[::-1]
```

6.4. Anagrama

Dos palabras son anagramas si tienen las mismas letras. Counter(a) == Counter(b) también sirve.

```
anagram.py

def is_anagram(a, b):
    return sorted(a) == sorted(b)
```

7. Grafos

7.1. Leer un grafo

Un grafo tiene n nodos y m aristas. Lo guardamos como lista de adyacencia: $g[u]$ es la lista de vecinos de u . En la entrada los nodos suelen ir de 1 a n , así que restamos 1. Si el grafo es dirigido (aristas de un solo sentido), borra la última línea. Un árbol es un grafo conexo con $m = n - 1$.

read-graph.py

```
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
```

7.2. Leer un grafo con pesos

Cada arista tiene un coste w ; guardamos pares (vecino, coste).

read-weighted-graph.py

```
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v, w = map(int, input().split())
    g[u - 1].append((v - 1, w))
    g[v - 1].append((u - 1, w))
```

7.3. BFS (camino más corto)

Visita los nodos por orden de distancia desde s . $dist[v]$ es el mínimo número de aristas de s a v , o -1 si no se puede llegar. Solo para grafos sin pesos. $O(n + m)$.

bfs.py

```
from collections import deque

def bfs(g, s):
    dist = [-1] * len(g)
```

```
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

7.4. DFS (alcanzabilidad)

$seen[v]$ es True si se puede llegar a v desde s . Usa una pila en vez de recursión para evitar RecursionError. $O(n + m)$.

dfs.py

```
def dfs(g, s):
    seen = [False] * len(g)
    seen[s] = True
    stack = [s]
    while stack:
        u = stack.pop()
        for v in g[u]:
            if not seen[v]:
                seen[v] = True
                stack.append(v)
    return seen
```

7.5. Componentes conexas

Número de grupos de nodos conectados entre sí. Lanza un DFS desde cada nodo que aún no se ha visto.

components.py

```
def count_components(g):
    seen = [False] * len(g)
    count = 0
    for s in range(len(g)):
        if seen[s]:
            continue
```

```

count += 1
seen[s] = True
stack = [s]
while stack:
    u = stack.pop()
    for v in g[u]:
        if not seen[v]:
            seen[v] = True
            stack.append(v)

return count

```

7.6. BFS en una cuadrícula

Cada celda es un nodo unido a sus 4 vecinas; '#' es una pared. `dist[r][c]` es el mínimo número de pasos desde `(sr, sc)`, o `-1` si no se puede llegar.

```

grid-bfs.py

from collections import deque

def grid_bfs(grid, sr, sc):
    n, m = len(grid), len(grid[0])
    dist = [[-1] * m for _ in range(n)]
    dist[sr][sc] = 0
    q = deque([(sr, sc)])
    while q:
        r, c = q.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            x, y = r + dr, c + dc
            if not (0 <= x < n and 0 <= y < m):
                continue
            if grid[x][y] == "#" or dist[x][y] != -1:
                continue
            dist[x][y] = dist[r][c] + 1
            q.append((x, y))
    return dist

```